Lending Club Loan Default Prediction: A Four-Phase Pipeline from Pandas to Conversational AI

Aayush Nepal, Junwei Zhang, Lusi Zhang
404 Team Not Found • EAS 587 Spring 2026
University at Buffalo, Buffalo, NY, USA

***Abstract*—*In this project, we build a loan default prediction system for Lending Club loans from 2014 to 2017. We use the same problem in four different ways: first with a local pandas pipeline, then with a tuned scikit-learn model, then with a Spark and Delta Lake medallion structure that also adds Federal Reserve macro data, and finally with a Streamlit app and a Model Context Protocol (MCP) server. The final XGBoost model gets 0.726 AUC-ROC, 0.410 AUC-PR, and a Kolmogorov–Smirnov statistic of 0.328 on the held-out 2017 test set with 314,212 loans. The main things that helped the result were the 21-column leakage filter, the chronological train/validation/test split, and the custom eight-stage preprocessing pipeline. This pipeline is only fit on the training data and then reused in the app and MCP server. We also tested whether seven extra Federal Reserve features improve the model. The validation AUC only changed by 0.0005, so we mainly use the macro layer for business insights instead of claiming a model improvement.*

***Index Terms*—*loan default prediction, gradient boosting, leakage discipline, Spark, Delta Lake, Model Context Protocol, scikit-learn pipelines.*

I. INTRODUCTION AND BACKGROUND

Lending Club is an unsecured personal-loan marketplace, and it issued more than two million loans from 2007 to 2020. For every application, Lending Club gives a sub-grade from A1, which means lower risk, to G5, which means higher risk. From the lender side, the main question is simple: will the borrower fully repay the loan, or will the loan become charged off? Because these loans are unsecured, the loss can be serious when a default happens. Phase 4 is the final part of our semester project on this question.

This paper is not only about training one model. We use the same loan default prediction task across four phases. Phase 1 builds the first data-cleaning work in pandas. Phase 2 trains and tunes the scikit-learn pipeline, and the XGBoost model becomes our production model. Phase 3 rebuilds the data layer in Spark and Delta Lake, and also adds macroeconomic data from the Federal Reserve. Phase 4 uses the same trained model in two deployment settings: a Streamlit app and an MCP tool in Claude Desktop. We tried to reuse earlier artifacts instead of starting again in each phase. Fig. 1 shows this overall pipeline.

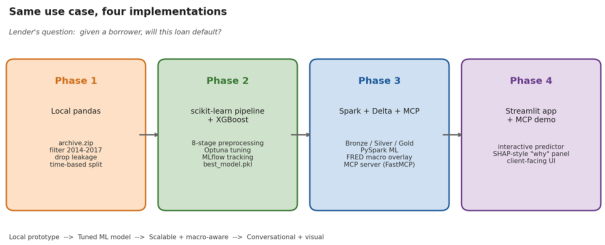

Fig. 1. Same use case, four implementations. Each phase reuses the artifacts of the previous one; only the surface changes.

The work draws on three lines of related work. Tree-based gradient boosting [1] remains the most consistently strong family for tabular credit data, with XGBoost [2] the modern reference implementation and SHAP [3] the standard tool for per-prediction interpretability. Hyperparameter optimisation via Optuna's TPE sampler [4] is the validation-driven equivalent of Spark MLlib's `CrossValidator`. Macro-augmented credit scoring is a well-known generalisation: borrower-level features explain cross-sectional default rates well but miss the time-varying component that arrives with the business cycle. We use FRED [5] as the source for unemployment, federal funds rate, and CPI data.

II. DATA DESCRIPTION

A. Source and Scope

The primary dataset is Ethon Liu's Kaggle release of Lending Club issuance data covering 2007Q1 through 2020Q3 [6], totalling roughly 2.9 million loans before filtering. We restrict the study to loans issued between January 2014 and December 2017 for two reasons: pre-2014 issuance volume is too sparse to be cross-sectionally representative, and loans issued after 2017 are not fully matured during the study window so the `loan_status` label is unreliable. Filtering to terminal-status records (`Fully Paid` or `Charged Off`) yields 1,355,679 completed loans across the four issuance years.

B. Target and Class Balance

The target y is the binary indicator y = 1 if `loan_status` equals `Charged Off`, otherwise 0. The empirical default rate across the full filtered cohort is approximately 21%, giving a paid-to-default ratio of approximately 4:1. We do not apply SMOTE or any other resampling. Tree-based learners handle the imbalance natively through the `scale_pos_weight`

parameter and in our experiments resampling neither improved AUC-ROC nor AUC-PR while discarding training data.

C. Time-Based Splitting

We use a time-based split because Lending Club default rates can change with the macro economy. If we randomly shuffle the data, future-period information can indirectly get into the training set through the class balance. We instead split chronologically by issue date: **train** is the first 80% of 2014–2016 issuances by `issue_d`; **validation** is the remaining 20% of 2014–2016; and **test** is the entire 2017 cohort, n = 314,212 loans, never observed during training, hyperparameter selection, or threshold tuning.

D. Leakage Discipline

One important Phase 1 decision is the manual drop list of 21 columns. These columns are only known after the loan already has payment history or an outcome, so using them would create target leakage. Many public Lending Club notebooks keep some of these columns and then get very high but unrealistic scores. Table I lists the columns we removed.

TABLE I. Twenty-one post-loan columns dropped to prevent target leakage.

Category	Columns
Payment history	total_pymnt, total_pymnt_inv, total_rec_prncp, total_rec_int, total_rec_late_fee, last_pymnt_amnt, last_pymnt_d
Outstanding principal	out_prncp, out_prncp_inv
Recovery / collection	recoveries, collection_recovery_fee, last_credit_pull_d
Settlement / hardship	hardship_flag, debt_settlement_flag, debt_settlement_flag_date, settlement_status, settlement_date, settlement_amount, settlement_percentage, settlement_term
Updated credit signal	last_fico_range_low, last_fico_range_high

E. Secondary Data: Federal Reserve Macro Series

The Phase 4 rubric asks us to add recommended secondary data sources. We pull eight FRED series at fit time using `pandas-datareader`: civilian unemployment rate (`UNRATE`), federal funds effective rate (`FEDFUNDS`), the all-urban consumer price index (`CPIAUCSL`), and unemployment for the five highest-volume states (`CAUR`, `TXUR`, `NYUR`, `FLUR`, `ILUR`). From these we engineer four derived features: 12-month CPI change as an inflation proxy, real interest rate as int*rate* − *CPI*YOY, the spread of borrower rate over the federal funds rate, and a per-borrower state unemployment lookup. The join key is the loan's issue (year, month) for national series and (year, month, addr_state) for state series.

III. METHODOLOGY

A. Phase 1: Local Cleaning

Phase 1 runs as a sequence of nine numbered scripts in `src/`, each consuming the previous one's CSV and writing the next. Cleaning drops any column with more than 30% missing values, applies the leakage filter (Table I), removes uninformative columns (free-text fields, identifiers, near-duplicates, near-zero-variance columns), and normalises numeric strings (e.g. stripping the percent sign on `int_rate` and the unit suffix on `term`). Date columns (`issue_d`, `earliest_cr_line`) are parsed to `datetime`.

B. Phase 2: Eight-Stage Preprocessing Pipeline

The Phase 2 pipeline is a single `sklearn.Pipeline` of eight custom `BaseEstimator`/`TransformerMixin` stages, fit on training data only and applied identically to the validation and test splits. The stages execute in fixed order and Fig. 2 visualises the flow.

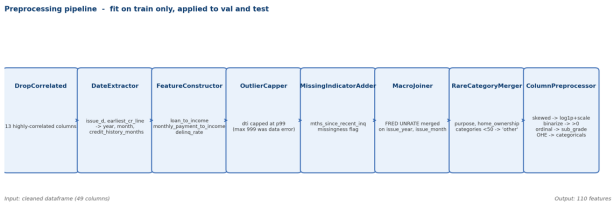

Fig. 2. Eight-stage preprocessing pipeline. The output is a 110-column dense feature matrix consumed by every downstream model.

(1) DropCorrelated removes 13 highly-correlated columns identified in EDA (e.g. `fico_range_high` which is r ≈ 1.0 with `fico_range_low`).

(2) DateExtractor splits `issue_d` and `earliest_cr_line` into `issue_year`, `issue_month`, and `credit_history_months`.

(3) FeatureConstructor engineers three risk-relevant ratios: loan-to-income, monthly-payment-to-income (using the standard PMT formula on the borrower's interest rate and term), and delinquency rate.

(4) OutlierCapper clips `dti` at the 99th percentile to neutralise a known data-entry error (the raw maximum is 999).

(5) MissingIndicatorAdder attaches a binary `_missing` flag to `mths_since_recent_inq` before imputation, because missingness of inquiry data is informative.

(6) MacroJoiner fetches FRED `UNRATE` and merges it on issue (year, month); if the FRED endpoint is unreachable, the stage falls back to a constant 5.0% so the column survives `SimpleImputer`.

(7) RareCategoryMerger collapses `purpose` and `home_ownership` categories with fewer than 50 training observations into an "other" bucket.

(8) ColumnPreprocessor routes columns into five buckets: skewed numerics (log-1p then standardise), binarise (cast to indicator-of-positive), ordinal `sub_grade` (custom A1<...<G5 ordering then standardise), one-hot for low-

cardinality categoricals, and standardised for the residual numerics.

A subtle but consequential implementation detail is that the pickled pipeline references custom-class symbols under module path `__main__`, because the script that fits and pickles it runs as `__main__`. Both downstream consumers (the Streamlit app and the MCP server) inject those symbols into `__main__` before unpickling, as shown in Listing 1.

Listing 1. Pickle-time symbol injection used by every consumer of preprocessing_pipeline.pkl.

```
import __main__, importlib.util, pickle
spec = importlib.util.spec_from_file_location(
    "_pipeline_module",
    ROOT / "src" / "06_data_processing_pipeline.py",
)
module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(module)
for name, obj in vars(module).items():
    if not name.startswith("__"):
        setattr(__main__, name, obj)
with open(PIPELINE_PATH, "rb") as f:
    pipeline = pickle.load(f)
```

C. Phase 2: Model Selection

We compare four classifiers on the validation set: ℓ_2-regularised Logistic Regression as a baseline, a Decision Tree, an XGBoost [2] classifier, and scikit-learn's `HistGradientBoostingClassifier` [7]. The three tree models receive 50 Optuna [4] trials each with the TPE sampler optimising validation AUC-ROC. All trials are tracked in MLflow [8]; class imbalance is handled via `scale_pos_weight` for XGBoost and `sample_weight` for HGB. The selection criterion is validation AUC-ROC, with KS as a tie-breaker, and the operating threshold for downstream evaluation is chosen on the validation set by Youden's J statistic [9]: $\hat{\tau} = \text{argmax}_\tau\ (\text{TPR}(\tau) - \text{FPR}(\tau))$.

D. Phase 3: Spark + Delta Lake Medallion

Phase 3 is a parallel Databricks rebuild that re-implements Phases 1 and 2 on Spark [10] with Delta Lake [11] storage. It follows the standard medallion pattern: **Bronze** (`bronze_loans`) ingests the raw Kaggle dump as-is; **Silver** (`silver_loans`) applies the same cleaning and leakage filter as Phase 1; **Gold** (`gold_loans_train`, `_val`, `_test`) materialises the time-based split, with stratified 200,000 / 50,000 samples for train and validation and the full 2017 cohort retained for test.

For model fitting in Phase 3, we still use pandas and scikit-learn instead of Spark MLlib. The reason is practical: in Databricks Free Edition, the serverless runtime blocks several PySpark ML feature-engineering classes through a Py4J access policy, including `Imputer`, `StringIndexer`, `OneHotEncoder`, and `VectorAssembler`. These are exactly the classes we would need for an MLlib pipeline. So Spark is used for the Bronze, Silver, and Gold data layers, while the model training remains in scikit-learn.

E. Phase 3: Macro Augmentation Experiment

A separate notebook checks whether the seven extra macro features, beyond the production `unemployment_rate` already added by `MacroJoiner`, improve predictive performance. We re-fit the tuned Logistic Regression model on the Gold tables twice: once with only loan-level features and once with the seven macro features added. Then we compare validation AUC-ROC.

F. Phase 4: Deployment Surfaces

In Phase 4, we reused the trained `best_model.pkl` and `preprocessing_pipeline.pkl` files without changing them. These artifacts are exposed through two deployment surfaces.

The first surface is an interactive Streamlit application [12], located at `src/app/streamlit_app.py`. The app organizes the project into nine tabs and works as both a presentation tool and a live demo. It includes a threshold tuner, a compare mode that scores two loan profiles side by side, and prediction explanations using XGBoost's native pred_contribs. This provides SHAP-style contribution values [3] without needing an extra SHAP dependency.

The second surface is a Model Context Protocol server, located at `src/mcp/server.py`, built with FastMCP [13]. The server provides one tool, `predict_loan_default`, with typed input parameters. It validates the user inputs, fills 24 secondary fields using population median or mode defaults, applies the same `PIPELINE.transform()` step used during training, and returns the default probability, risk tier, and recommendation text. With this setup, an analyst can describe a borrower in natural language inside Claude Desktop, and the LLM can call the model directly for what-if analysis.

Listing 2. MCP tool registration. Type annotations are required so FastMCP deserialises JSON-RPC arguments before they hit the validation block.

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("lending-default-predictor")

@mcp.tool()
def predict_loan_default(
    loan_amnt: float, term: int, int_rate: float,
    sub_grade: str, annual_inc: float, dti: float,
    fico_score: int, home_ownership: str, purpose: str,
    open_acc: int = 10, revol_util: float = 30.0,
    total_acc: int = 20, delinq_2yrs: int = 0,
    pub_rec: int = 0, inq_last_6mths: int = 0,
    verification_status: str = "Not Verified",
    earliest_cr_line: str = "2010-01",
) -> dict:
    ...  # validation, transform, predict_proba
```

IV. RESULTS AND ANALYSIS

A. Validation Bake-Off

Table II compares the validation results for the four candidate models. XGBoost performed slightly better than HistGradientBoosting on AUC-ROC and KS, while their AUC-PR scores were almost the same. XGBoost also trained much faster, so we selected it as the final production model.

TABLE II. Validation performance, 50 Optuna trials per tuned model.

Model	AUC-ROC	AUC-PR	KS	Fit (s)
Logistic Regression	0.7121	0.3757	0.306	8
Decision Tree (tuned)	0.7010	0.3572	0.290	908
HistGradientBoosting	0.7233	0.3933	0.322	2,633
XGBoost (winner)	**0.7238**	**0.3931**	**0.325**	**507**

B. Held-Out Test Set

We evaluated the final XGBoost model one time on the 2017 holdout test set, which contains 314,212 loans. This test set was not used during training, hyperparameter tuning, or threshold selection. The model achieved an AUC-ROC of 0.726, an AUC-PR of 0.410, and a KS statistic of 0.328. Since the default rate in the test set is about 0.210, the AUC-PR shows about 1.95 times improvement over the baseline class rate. Fig. 3 shows the ROC curve on the test set, with the Youden threshold marked. Fig. 4 shows the confusion matrix at this threshold, where $\tau = 0.495$. At this point, the model has 34.2% precision and 67.3% recall for charged-off loans. The recall result is especially useful for this project because the model can identify about two-thirds of future charge-offs before they happen. The trade-off is that some fully paid loans are also flagged for manual review.

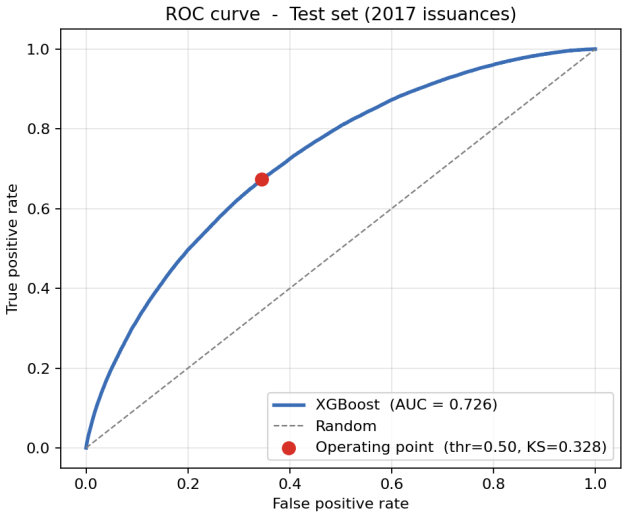

Fig. 3. Test ROC curve. Operating point selected by Youden's J on the validation set.

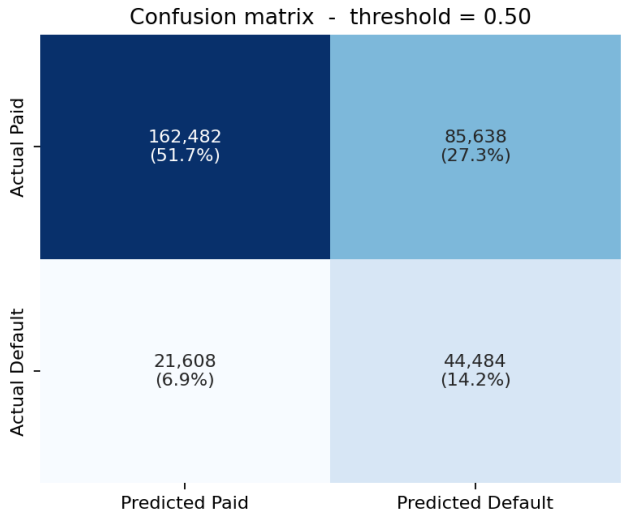

Fig. 4. Test confusion matrix at $\tau = 0.495$. Default recall 67.3% at precision 34.2%.

C. Feature Importance

Fig. 5 shows the top fifteen features ranked by XGBoost gain. The most important feature is `sub_grade`, which has about three times more gain than the next feature. This result makes sense because Lending Club's sub-grade is already a risk-based score, so it contains a large amount of borrower risk information. Other important features include loan term, interest rate, home ownership, and recent credit account activity.

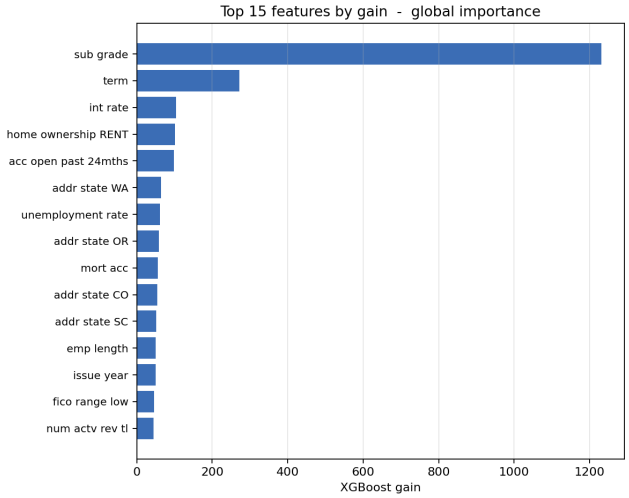

Fig. 5. Top-15 features by gain, XGBoost.

D. FRED Macro Augmentation

The macro experiment gave us three useful insights, but the results were not always simple. First, the default rate did not always go up when the national unemployment rate was higher. In our 2014-2017 data, some loans issued during lower-unemployment periods still had higher default rates, which may be related to looser lending standards during stronger economic periods. Second, most sub-grades had higher default rates when the Fed Funds rate was higher. This

suggests that riskier borrowers may be more affected by tighter monetary conditions. Third, the state unemployment difference showed mixed results across the top five states. Because of this, state-level unemployment is useful for economic context, but it was not a strong standalone predictor in our model.

V. DISCUSSION

A. What Worked

One of the most important parts of this project was keeping the data pipeline clean and realistic. The leakage filter helped us remove columns that would not be available before a loan outcome is known. Without removing those columns, the model could get unrealistically high scores by using future information.

The time-based split also worked well. Instead of randomly splitting the data, we trained on earlier loans and tested on 2017 loans. This is closer to a real-world setting, where a model is trained on past data and then used to predict future loans.

The preprocessing pipeline was another strong part of the project. All feature engineering, missing-value handling, encoding, scaling, and macro joining were placed inside one scikit-learn pipeline. This made the process more reproducible and also helped avoid differences between training and deployment.

XGBoost was a good final model choice. It gave the best validation AUC-ROC and trained much faster than HistGradientBoosting. It also supports feature contribution values, which made it easier to explain individual predictions in the Streamlit app..

B. What Did Not Work

The macroeconomic experiment had mixed results. Adding extra FRED macro features did not improve validation AUC. The model with loan features alone had a validation AUC of 0.7136, while the model with macro features had 0.7131. This means the additional macro variables were useful for analysis and explanation, but they did not improve prediction performance in our experiment.

In Phase 3, we also planned to use Spark MLlib for the full modeling pipeline. However, Databricks Free Edition blocked several needed MLlib feature tools. Because of this, we used Spark and Delta Lake for the data engineering part, but used pandas and scikit-learn for model fitting.

C. Limitations

This project still has several limitations. First, we did not complete a fairness audit. Since this is a credit-risk problem, fairness and compliance would be very important before any real-world use.

Second, the model was only tested on the 2017 holdout set. We did not test it on later periods, such as the COVID-era lending environment. Because of this, we cannot fully know how stable the model would be under very different economic conditions.

Third, the public Lending Club dataset does not include all information that a real lender might use. More detailed credit bureau data, borrower behavior data, and compliance-related variables could improve the model or help evaluate fairness.

Finally, the threshold was selected using Youden's J statistic. This is useful for balancing true positive and false positive rates, but it does not include the real financial cost of approving a risky loan or rejecting a good borrower.

C. Future Work

Future work should focus on making the system more production-ready. A fairness and adverse-impact analysis should be added before using the model for any real credit decision. The model should also be tested on newer loan cohorts to check whether performance changes over time.

Another useful improvement would be a cost-sensitive threshold. Instead of using a general statistical threshold, the cutoff could be chosen based on the actual business cost of defaults and false rejections.

A final extension would be to build a REST API for batch loan scoring. This would use the same saved preprocessing pipeline and model, similar to the MCP server, but would be easier to connect with other software systems.

VI. CONCLUSION

This project built a complete loan default prediction system using Lending Club data from 2014 to 2017. Across four phases, we developed a local pandas pipeline, trained and selected machine-learning models, rebuilt the data layer with Spark and Delta Lake, added FRED macroeconomic data, and deployed the final model through Streamlit and an MCP server.

The final XGBoost model achieved an AUC-ROC of 0.726 and an AUC-PR of 0.410 on the held-out 2017 test set with 314,212 loans. These results show that the model can rank loan default risk better than the baseline class rate, especially for identifying charged-off loans. The biggest lesson from the project is that careful data design is just as important as model selection. Removing leakage columns, using a chronological split, fitting preprocessing only on training data, and reusing the same pipeline during deployment all helped make the results more reliable.

The macroeconomic extension did not improve validation AUC, but it still added useful context. It helped us understand how default risk changes across economic conditions and showed that borrower-level features remain the strongest predictors in this dataset.

Overall, the project moved from raw data to a working prediction system. The final result is not only a trained model, but also a reproducible pipeline and deployment structure that could be extended for future credit-risk analysis.

VII. REFERENCES

[1] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189--1232, 2001.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785--794.

[3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Adv. Neural Inf. Process. Syst. 30, 2017, pp. 4765--4774.

[4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2019, pp. 2623--2631.

[5] Federal Reserve Bank of St. Louis, "FRED Economic Data," 2026. [Online]. Available: https://fred.stlouisfed.org/

[6] E. Liu, "Lending Club loan data 2007--2020Q3," Kaggle, 2020. [Online]. Available: https://www.kaggle.com/datasets/ethon0426/lending-club-20072020q1

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825--2830, 2011.

[8] Databricks, "MLflow: An open source platform for the machine learning lifecycle," 2024. [Online]. Available: https://mlflow.org/

[9] W. J. Youden, "Index for rating diagnostic tests," Cancer, vol. 3, no. 1, pp. 32--35, 1950.

[10] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," Commun. ACM, vol. 59, no. 11, pp. 56--65, 2016.

[11] M. Armbrust et al., "Delta Lake: High-performance ACID table storage over cloud object stores," Proc. VLDB Endow., vol. 13, no. 12, pp. 3411--3424, 2020.

[12] Snowflake Inc., "Streamlit: A faster way to build and share data apps," 2024. [Online]. Available: https://streamlit.io/

[13] Anthropic, "Model Context Protocol specification," 2024. [Online]. Available: https://modelcontextprotocol.io/